AQI Monitor Firmware Source Code Report

This report explains the code in ‘src/‘, ‘include/‘, and ‘platformio.ini‘.
It assumes you know web development and some Python, but are new to C, C++, Arduino firmware, and header files.

1. Big Picture

This project is firmware for an ESP8266 D1 Mini air quality monitor. Firmware is software that runs directly on a microcontroller instead of inside a browser, Node.js, or a desktop operating system.

The hardware pieces are:

- ESP8266 D1 Mini: the small Wi-Fi microcontroller board that runs the code.
- PMS5003: a particulate matter sensor. It reports PM1.0, PM2.5, PM10, and particle counts.
- MH-Z19E: a CO2 sensor. It reports carbon dioxide in ppm and a rough internal temperature.
- 2.4 inch SPI TFT display: the screen.
- Push button on D0: switches between display screens.

The firmware does four major jobs:

1. Read the PMS5003 particle sensor over serial.
2. Read the MH-Z19E CO2 sensor over serial.
3. Convert particle readings into US EPA AQI numbers and categories.
4. Draw a dashboard/details UI on the TFT screen.

The project is organized like this:

```
platformio.ini       Build configuration for PlatformIO.
include/config.h     Shared pin numbers, baud rates, timings, and options.
src/main.cpp         Main Arduino setup/loop and sensor scheduling.
src/pms5003.h/.cpp   Driver for the PMS5003 particle sensor.
src/mhz19.h/.cpp     Driver for the MH-Z19E CO2 sensor.
src/aqi.h/.cpp       AQI math and color/category helpers.
src/ui.h/.cpp        Display data model and TFT drawing code.
```

2. Arduino and C++ Basics

‘.cpp‘ files

‘.cpp‘ files contain C++ implementation code. In web terms, think of them like JavaScript or TypeScript modules that contain the actual function bodies.

Example:

```
int aqiFromPm25(float ugm3) { return interpolate(PM25_BP, 6, ugm3); }
```

This defines a function named ‘aqiFromPm25‘.

‘.h‘ files

‘.h‘ files are header files. They usually declare the public shape of code: functions, classes, structs, constants, and types that other files are allowed

to use.

In web-dev terms, a header file is loosely like a TypeScript ‘.d.ts‘ file plus an export list. It tells other files "these functions/classes exist", while the ‘.cpp‘ file says "here is how they work".

Example from ‘aqi.h‘:

```
int aqiFromPm25(float ugm3);
```

That line is a declaration. It does not contain the body. The matching body is in ‘aqi.cpp‘.

‘#include‘

‘#include‘ copies declarations from another file into the current file before compilation. It is closer to Python ‘import‘ or JavaScript ‘import‘, but more literal: the C/C++ preprocessor inserts the contents.

Examples:

```
#include <Arduino.h>
#include "aqi.h"
```

Angle brackets usually mean "library/framework header". Quotes usually mean "local project header".

‘#pragma once‘

Header files in this project start with:

```
#pragma once
```

This prevents the same header from being included multiple times in one compile unit. Without it, duplicate declarations can cause compiler errors.

‘setup()‘ and ‘loop()‘

Arduino programs use two special functions:

- ‘setup()‘ runs once when the board boots.
- ‘loop()‘ runs repeatedly forever.

This is like having an app initialization function, then an infinite event loop. There is no normal ‘main()‘ written by you; the Arduino framework provides it and calls your ‘setup()‘ and ‘loop()‘.

Integer types such as ‘uint8_t‘, ‘uint16_t‘, ‘uint32_t‘

Firmware cares about exact sizes because memory is limited and hardware protocols use byte-level formats.

- ‘uint8_t‘: unsigned 8-bit integer, 0 to 255. One byte.
- ‘uint16_t‘: unsigned 16-bit integer, 0 to 65535. Two bytes.
- ‘uint32_t‘: unsigned 32-bit integer, 0 to about 4.29 billion. Four bytes.

The ‘u‘ means unsigned, so negative values are not allowed.

‘constexpr‘

‘constexpr‘ defines a compile-time constant.

Example:

```
constexpr uint32_t SAMPLE_INTERVAL_MS = 5000;
```

This is like a ‘const‘ value in JavaScript, but stronger: the compiler knows it can treat the value as fixed while building the firmware.

Classes and structs

A ‘struct‘ is a group of fields. In this project, structs are mostly simple data containers.

Example:

```
struct PmsData {
    uint16_t pm1_std, pm25_std, pm10_std;
};
```

A ‘class‘ groups data and functions together. In this project, each sensor driver is a class.

Example:

```
class Pms5003 {
public:
    bool poll();
private:
    uint8_t _buf[32];
};
```

‘public‘ members can be used by other files. ‘private‘ members are internal to the class.

References: ‘Stream& serial‘

The ‘&‘ in ‘Stream&‘ means a reference. A reference is like passing an object by alias instead of copying it.

The sensor drivers receive a ‘Stream&‘ so they can work with any Arduino stream object that supports ‘read()‘, ‘write()‘, and ‘available()‘. Here that stream is a ‘SoftwareSerial‘ object.

The anonymous namespace

Several ‘.cpp‘ files contain:

```
namespace {
    ...
}
```

This is an unnamed namespace. It makes the things inside private to that ‘.cpp‘ file. In JavaScript terms, it is like variables/functions that are not exported from a module.

3. ‘platformio.ini‘

‘platformio.ini‘ is the PlatformIO build configuration. It tells PlatformIO what board, framework, libraries, serial speed, upload port, and compiler flags to use.

File comments

Lines beginning with ‘;‘ are comments in INI files. They are documentation for humans and are ignored by PlatformIO.

The top comment says the target hardware is:

```
ESP8266 D1 Mini + PMS5003 + MH-Z19E + 2.4 inch SPI TFT
```

It also explains three common commands:

```
pio run
pio run -t upload
pio device monitor
```

- ‘pio run‘: compile the firmware.
- ‘pio run -t upload‘: compile and flash it to the board.
- ‘pio device monitor‘: open the serial monitor to see logs.

Display driver note

The comments explain that the delivered display behaves like an ST7789V controller, not an ILI9341. Symptoms of using the wrong driver were ignored rotation, an 80px noise band, and swapped red/blue colors.

That is why the default environment is ST7789. There is also a fallback environment for ILI9341 displays.

‘[common]‘

This section defines shared build flags used by multiple environments.

```
[common]
base_flags =
```

‘base_flags‘ is a named list of compiler flags. Other environments reuse it via ‘${common.base_flags}‘.

‘-DUSER_SETUP_LOADED=1‘

Flags beginning with ‘-D‘ define preprocessor macros. They are compile-time values.

This one tells ‘TFT_eSPI‘ that the project is providing display setup through compiler flags, so you do not need to edit the library’s ‘User_Setup.h‘.

That is useful because library files should usually stay untouched. Project configuration belongs in the project.

TFT pin flags

```
-DTFT_MISO=12
-DTFT_MOSI=13
-DTFT_SCLK=14
```

```
-DTFT_CS=15
-DTFT_DC=0
-DTFT_RST=-1
```

These define the ESP8266 GPIO pins used by the display.

- MISO is GPIO12 / D6. It is defined for the SPI bus, but the display SDO is not wired.
- MOSI is GPIO13 / D7.
- SCLK is GPIO14 / D5.
- CS is GPIO15 / D8.
- DC is GPIO0 / D3.
- RST is ‘-1‘, meaning the TFT reset is not controlled by a separate GPIO. It is tied to the D1 Mini reset pin.

These names are required by the ‘TFT_eSPI‘ library at compile time.

Font flags

```
-DLOAD_GLCD=1
-DLOAD_FONT2=1
-DLOAD_FONT4=1
-DLOAD_FONT6=1
-DLOAD_FONT7=1
-DLOAD_FONT8=1
-DLOAD_GFXFF=1
```

These tell ‘TFT_eSPI‘ which built-in fonts to include in the firmware. Fonts consume flash space, so embedded projects often include only the fonts they need.

The UI code later asks for fonts by number:

```
tft.drawString("US AQI", x, y, 2);
tft.drawNumber(model.aqi, x, y, 6);
```

The final ‘2‘ or ‘6‘ is the font number.

SPI speed flags

```
-DSPI_FREQUENCY=27000000
-DSPI_READ_FREQUENCY=20000000
```

These set display SPI speeds. SPI is the communication bus used by the screen. The comment says 27 MHz is safe on jumper wires. Faster SPI means faster screen updates, but long wires and breadboards can become unreliable at high speeds.

‘[env]‘

This section contains settings common to all PlatformIO environments.

```
platform = espressif8266
board = d1_mini
framework = arduino
```

These say:

- Use the ESP8266 platform package.
- Build for the Wemos/Lolin D1 Mini board profile.

- Use the Arduino framework.

```
monitor_speed = 115200
```

This is the baud rate used by the serial monitor. It must match ‘DEBUG_BAUD‘ in ‘include/config.h‘.

```
monitor_filters = esp8266_exception_decoder, default
```

The exception decoder makes ESP8266 crash dumps more readable in the serial monitor.

```
upload_speed = 460800
upload_port = COM6
```

These configure flashing speed and the Windows serial port.

```
lib_deps =
    bodmer/TFT_eSPI@^2.5.43
```

This declares one library dependency: ‘TFT_eSPI‘, version compatible with 2.5.43. PlatformIO downloads and builds it automatically.

‘[env:d1_mini]‘

This is the default build environment.

```
[env:d1_mini]
build_flags =
    ${common.base_flags}
    -DST7789_DRIVER=1
    -DTFT_WIDTH=240
    -DTFT_HEIGHT=320
    -DTFT_INVERSION_OFF=1
    -DTFT_RGB_ORDER=TFT_BGR
```

It uses all shared flags, then selects the ST7789 display driver.

The display is physically 240x320 pixels, but the UI rotates it to landscape, so the code draws as 320x240.

‘TFT_INVERSION_OFF‘ fixes photo-negative-looking colors on this panel. ‘TFT_RGB_ORDER=TFT_BGR‘ fixes red/blue channel order.

‘[env:d1_mini_ili9341]‘

This fallback environment selects the ILI9341 driver instead:

```
[env:d1_mini_ili9341]
build_flags =
    ${common.base_flags}
    -DILI9341_DRIVER=1
```

Use it only if the screen is genuinely ILI9341.

4. ‘include/config.h‘

This header stores shared configuration used by multiple ‘.cpp‘ files.

It starts with:

```
#pragma once
#include <Arduino.h>
```

‘Arduino.h‘ provides Arduino types and functions such as ‘uint32_t‘, ‘HIGH‘, ‘LOW‘, ‘pinMode‘, ‘digitalRead‘, and ‘millis‘.

Pin assignments

```
constexpr int PIN_PMS_RX = 4;
constexpr int PIN_MHZ_RX = 5;
constexpr int PIN_MHZ_TX = 2;
constexpr int PIN_BUTTON = 16;
```

These are ESP8266 GPIO numbers, not just the printed D labels. The comments map them to D1 Mini labels:

- ‘PIN_PMS_RX = 4‘: D2 receives data from PMS5003 TXD.
- ‘PIN_MHZ_RX = 5‘: D1 receives data from MH-Z19E TX.
- ‘PIN_MHZ_TX = 2‘: D4 sends commands to MH-Z19E RX.
- ‘PIN_BUTTON = 16‘: D0 reads the screen-toggle button.

The PMS5003 connection is RX only because this firmware listens to the PMS5003 in active mode. It does not send commands to it.

The MH-Z19E needs both RX and TX because the firmware sends a request command and waits for a response.

The button uses GPIO16 / D0 because GPIO16 has an internal pulldown. That means the button should connect D0 to 3.3V, and pressed reads as ‘HIGH‘.

Serial settings

```
constexpr uint32_t DEBUG_BAUD = 115200;
constexpr uint32_t PMS_BAUD   = 9600;
constexpr uint32_t MHZ_BAUD   = 9600;
```

The debug serial port talks to your computer at 115200 baud.

Both sensors use 9600 baud, fixed by the sensor protocols.

Timing settings

```
constexpr uint32_t SAMPLE_INTERVAL_MS = 5000;
```

The firmware attempts one full sensor cycle every 5 seconds.

```
constexpr uint32_t PMS_LISTEN_TIMEOUT_MS = 3500;
```

The PMS5003 sends frames periodically, around every 2.3 seconds in stable mode. The firmware listens up to 3.5 seconds before giving up for this cycle.

```
constexpr uint32_t MHZ_RESPONSE_TIMEOUT_MS = 500;
```

The MH-Z19E normally replies quickly, around 50 ms. The firmware waits up to 500 ms before treating the request as failed.

```
constexpr uint32_t DATA_STALE_MS = 30000;
```

If a valid reading is older than 30 seconds, the UI treats it as stale and shows missing/no-data state.

Warm-up settings

```
constexpr uint32_t PMS_WARMUP_MS = 30000;
constexpr uint32_t MHZ_WARMUP_MS = 60000;
```

Sensors need time after boot before their readings are reliable. The PMS5003 gets 30 seconds. The MH-Z19E gets 60 seconds.

The firmware can still read during warm-up, but the UI will not treat the data as valid until the warm-up time has passed.

MH-Z19E ABC setting

```
constexpr bool MHZ19_ABC_ENABLED = true;
```

ABC means Automatic Baseline Correction. The CO2 sensor assumes it sees fresh air, about 400 ppm, at least once per 24 hours and adjusts itself.

This is good in a normal home or office that gets ventilated. It can be wrong for greenhouses or always-occupied spaces that never return to fresh-air CO2 levels.

5. ‘src/aqi.h‘

This header declares the AQI-related functions and types.

```
enum AqiCategory : uint8_t {
    AQI_GOOD = 0,
    AQI_MODERATE,
    AQI_SENSITIVE,
    AQI_UNHEALTHY,
    AQI_VERY_UNHEALTHY,
    AQI_HAZARDOUS,
};
```

An ‘enum‘ is a named set of values. In web terms, think of a TypeScript union or Python enum. ‘: uint8_t‘ means store it as one byte.

The category values represent the standard AQI bands:

- Good
- Moderate
- Unhealthy for Sensitive Groups
- Unhealthy
- Very Unhealthy
- Hazardous

These function declarations follow:

```
int aqiFromPm25(float ugm3);
int aqiFromPm10(float ugm3);
```

They convert PM2.5 or PM10 concentration values into AQI numbers. The return value is ‘-1‘ if input is invalid.

```
AqiCategory aqiCategory(int aqi);
```

This converts a numeric AQI such as ‘73‘ into a category such as ‘AQI_MODERATE‘.

```
const char* aqiCategoryName(AqiCategory c);
```

This returns short text for the display, such as ‘"GOOD"‘ or ‘"MODERATE"‘.

‘const char*‘ is a C-style string. In firmware code it is common because it is lightweight compared with dynamic string objects.

```
uint16_t aqiCategoryColor(AqiCategory c);
```

This returns a 16-bit display color in RGB565 format. RGB565 is a compact color format commonly used by small TFT screens.

The last two declarations are for CO2 display labels and colors:

```
const char* co2Label(int ppm);
uint16_t co2Color(int ppm);
```

CO2 does not use the EPA AQI scale, so the code uses simple comfort bands.

6. ‘src/aqi.cpp‘

This file implements the AQI math and label/color helpers.

```
#include "aqi.h"
#include <TFT_eSPI.h>
```

It includes its own header and the display library header. The display header is needed for constants such as ‘TFT_GREEN‘, ‘TFT_YELLOW‘, and ‘TFT_RED‘.

‘Breakpoint‘

Inside the anonymous namespace:

```
struct Breakpoint { float cLo, cHi; int iLo, iHi; };
```

Each AQI breakpoint row maps a pollutant concentration range to an AQI index range.

Fields:

- ‘cLo‘: low concentration.
- ‘cHi‘: high concentration.
- ‘iLo‘: low AQI index.
- ‘iHi‘: high AQI index.

For example, PM2.5 from 0.0 to 9.0 ug/m3 maps to AQI 0 to 50.

PM2.5 breakpoints

```
const Breakpoint PM25_BP[] = {
    {0.0f, 9.0f, 0, 50},
    ...
};
```

This array contains the May 2024 revised EPA PM2.5 breakpoints. The ‘f‘ suffix

means the number is a ‘float‘, not a ‘double‘.

This matters on microcontrollers because ‘float‘ uses less memory.

PM10 breakpoints

```
const Breakpoint PM10_BP[] = {
    {0.0f, 54.0f, 0, 50},
    ...
};
```

This is the PM10 equivalent table.

‘interpolate‘

```
int interpolate(const Breakpoint* table, size_t n, float c)
```

This helper takes:

- ‘table‘: pointer to a breakpoint array.
- ‘n‘: number of rows in the table.
- ‘c‘: concentration value.

If ‘c < 0‘, it returns ‘-1‘, meaning invalid input.

Then it loops through the table:

```
for (size_t i = 0; i < n; i++) {
    const Breakpoint& b = table[i];
    if (c <= b.cHi) {
        ...
    }
}
```

‘size_t‘ is an unsigned integer type used for sizes and array indexes.

‘const Breakpoint& b = table[i];‘ creates a read-only reference to the current breakpoint row. It avoids copying the struct.

The interpolation formula is:

```
float aqi = (float)(b.iHi - b.iLo) / (b.cHi - b.cLo) * (c - b.cLo) + b.iLo;
```

This is a linear mapping from concentration range to AQI range.

In Python-like pseudocode:

```
aqi = ((i_hi - i_lo) / (c_hi - c_lo)) * (c - c_lo) + i_lo
```

Then:

```
return (int)(aqi + 0.5f);
```

This rounds to the nearest integer by adding 0.5 and truncating.

If the concentration is above the last breakpoint, the function returns 500.

AQI conversion functions

```
int aqiFromPm25(float ugm3) { return interpolate(PM25_BP, 6, ugm3); }
int aqiFromPm10(float ugm3) { return interpolate(PM10_BP, 6, ugm3); }
```

These are small wrappers around ‘interpolate‘.

The ‘6‘ is the number of breakpoint rows in each table.

‘aqiCategory‘

```
AqiCategory aqiCategory(int aqi)
```

This converts a number to a category using standard AQI thresholds:

- 0-50: good
- 51-100: moderate
- 101-150: sensitive
- 151-200: unhealthy
- 201-300: very unhealthy
- 301+: hazardous

One detail: if ‘aqi‘ is negative, it is also ‘<= 50‘, so this function would return ‘AQI_GOOD‘. The calling code only calls it for valid AQI values, so that is acceptable in this project.

‘aqiCategoryName‘

This ‘switch‘ returns display text for each category.

```
case AQI_GOOD: return "GOOD";
```

The default returns ‘"HAZARDOUS"‘. That covers both ‘AQI_HAZARDOUS‘ and any unexpected enum value.

‘aqiCategoryColor‘

This maps categories to TFT colors:

- Good: green
- Moderate: yellow
- Sensitive: orange
- Unhealthy: red
- Very unhealthy: purple
- Hazardous: maroon

CO2 helpers

```
const char* co2Label(int ppm)
```

CO2 bands:

- Below 800 ppm: ‘FRESH‘
- 800-1199 ppm: ‘OK‘
- 1200-1999 ppm: ‘STUFFY‘
- 2000+ ppm: ‘BAD‘

‘co2Color‘ uses the same thresholds and returns green, yellow, orange, or red.

7. ‘src/pms5003.h‘

This header declares the PMS5003 driver.

The PMS5003 sends binary frames over UART. A frame is 32 bytes:

```
0x42 0x4D + length + 13 data words + checksum
```

UART is serial communication. You can think of it as a byte stream, similar to reading bytes from a socket.

‘PmsData‘

```
struct PmsData {
    uint16_t pm1_std, pm25_std, pm10_std;
    uint16_t pm1_atm, pm25_atm, pm10_atm;
    uint16_t n03, n05, n10, n25, n50, n100;
};
```

This struct stores the parsed sensor reading.

The first three values are ‘"CF=1"‘ factory-standard particle values.

The next three are atmospheric-environment values. The firmware uses these for AQI because they are intended for real ambient air measurements.

The final six are particle counts per 0.1 liter of air:

- ‘n03‘: particles larger than 0.3 micrometers.
- ‘n05‘: particles larger than 0.5 micrometers.
- ‘n10‘: particles larger than 1.0 micrometers.
- ‘n25‘: particles larger than 2.5 micrometers.
- ‘n50‘: particles larger than 5.0 micrometers.
- ‘n100‘: particles larger than 10 micrometers.

‘class Pms5003‘

```
explicit Pms5003(Stream& serial) : _ser(serial) {}
```

This is the constructor. It receives a serial stream and stores it in ‘_ser‘.

‘explicit‘ prevents accidental conversions. It is a C++ safety habit.

```
void reset() { _idx = 0; }
```

This discards any half-read frame by resetting the buffer index.

```
bool poll();
```

This is the main parser function. It reads any bytes currently available. It returns ‘true‘ when a complete valid frame was parsed.

```
const PmsData& data() const { return _data; }
```

This returns the latest parsed data by read-only reference.

The private fields are:

```
Stream& _ser;
uint8_t _buf[32];
uint8_t _idx = 0;
PmsData _data{};
```

- ‘_ser‘: the serial stream.

- ‘_buf‘: 32-byte receive buffer.
- ‘_idx‘: current write position in the buffer.
- ‘_data‘: latest valid parsed reading.

The underscore prefix is a common C++ convention for private member variables.

8. ‘src/pms5003.cpp‘

This file implements the PMS5003 parser.

Constants

```
constexpr uint8_t START1 = 0x42;
constexpr uint8_t START2 = 0x4D;
constexpr uint8_t FRAME_LEN = 32;
constexpr uint16_t PAYLOAD_LEN = 28;
```

PMS5003 frames begin with two fixed bytes: hex ‘42 4D‘. The full frame is 32 bytes. The payload length field should say 28.

Hex numbers start with ‘0x‘. For example, ‘0x42‘ is decimal 66.

‘word16‘

```
uint16_t word16(const uint8_t* p) { return (uint16_t)p[0] << 8 | p[1]; }
```

The sensor sends 16-bit numbers as two bytes, high byte first. This function combines two bytes into one ‘uint16_t‘.

Example:

```
bytes: 0x01 0xF4
value: 500
```

The expression shifts the first byte left by 8 bits, then ORs in the second byte.

‘Pms5003::poll‘

‘Pms5003::poll‘ means "the ‘poll‘ method that belongs to the ‘Pms5003‘ class".

```
bool gotFrame = false;
```

The function starts assuming no valid frame was found.

```
while (_ser.available()) {
```

This loops while there are bytes waiting in the serial receive buffer.

```
uint8_t b = (uint8_t)_ser.read();
```

Read one byte. ‘_ser.read()‘ returns an integer, so it is cast to ‘uint8_t‘.

Frame synchronization

```
if (_idx == 0 && b != START1) continue;
```

If the parser is waiting for the first byte and this byte is not ‘0x42‘, ignore

it.

```
if (_idx == 1 && b != START2) { _idx = (b == START1) ? 1 : 0; continue; }
```

If the parser got the first start byte but the second byte is wrong, it resynchronizes.

The ternary expression:

```
(b == START1) ? 1 : 0
```

means "if ‘b‘ is another possible first start byte, keep ‘_idx‘ at 1; otherwise reset to 0".

This prevents missing a frame if the stream contains ‘42 42 4D‘.

Buffer fill

```
_buf[_idx++] = b;
if (_idx < FRAME_LEN) continue;
```

Store the byte, then increment ‘_idx‘. If the frame is not complete yet, keep reading.

Once 32 bytes are collected:

```
_idx = 0;
```

Reset for the next frame.

Length check

```
if (word16(_buf + 2) != PAYLOAD_LEN) continue;
```

‘_buf + 2‘ is pointer arithmetic. It means "start reading at byte index 2".
Bytes 2 and 3 contain the frame payload length.

If the length is not 28, the frame is rejected.

Checksum check

```
uint16_t sum = 0;
for (uint8_t i = 0; i < FRAME_LEN - 2; i++) sum += _buf[i];
if (sum != word16(_buf + 30)) continue;
```

The PMS5003 checksum is the sum of the first 30 bytes. The last two bytes store the expected checksum. If they do not match, the frame is corrupt and ignored.

Data extraction

The parser then maps byte positions into struct fields:

```
_data.pm1_std  = word16(_buf + 4);
_data.pm25_std = word16(_buf + 6);
...
_data.n100 = word16(_buf + 26);
```

Each sensor value is two bytes. The offsets come from the PMS5003 datasheet.

After storing all values:

```
gotFrame = true;
```

At the end, the function returns whether any valid frame was parsed.

9. ‘src/mhz19.h‘

This header declares the MH-Z19E CO2 sensor driver.

The MH-Z19 family uses 9-byte command and response frames:

```
0xFF 0x01 CMD D0 D1 D2 D3 D4 CHECKSUM
```

The important commands are:

- ‘0x86‘: read CO2 concentration.
- ‘0x79‘: turn ABC auto-calibration on/off.
- ‘0x87‘: zero-point calibration.

‘class Mhz19‘

The constructor is:

```
explicit Mhz19(Stream& serial) : _ser(serial) {}
```

Like the PMS driver, this receives an Arduino stream and stores it.

Public methods:

```
void requestReading();
bool poll();
void setAutoCalibration(bool enabled);
void calibrateZero();
int ppm() const;
int temperature() const;
```

- ‘requestReading‘: send a command asking for a CO2 reading.
- ‘poll‘: read bytes until a valid response is found.
- ‘setAutoCalibration‘: enable/disable ABC.
- ‘calibrateZero‘: tell the sensor the current air should be treated as 400 ppm. The comments correctly warn this should only be done after sufficient fresh-air exposure.
- ‘ppm‘: latest valid CO2 reading.
- ‘temperature‘: rough internal sensor temperature.

Private methods and fields:

```
void sendCommand(uint8_t cmd, uint8_t d0 = 0);
Stream& _ser;
uint8_t _buf[9];
uint8_t _idx = 0;
int _ppm = -1;
int _temp = 0;
```

‘d0 = 0‘ is a default parameter. If the caller does not pass the second argument, it defaults to zero.

10. ‘src/mhz19.cpp‘

This file implements MH-Z19E commands and response parsing.

Command constants

```
constexpr uint8_t CMD_READ_CO2  = 0x86;
constexpr uint8_t CMD_ABC       = 0x79;
constexpr uint8_t CMD_ZERO_CAL  = 0x87;
```

These are protocol command bytes.

Checksum function

```
uint8_t checksum(const uint8_t* frame) {
    uint8_t sum = 0;
    for (uint8_t i = 1; i < 8; i++) sum += frame[i];
    return (uint8_t)(0xFF - sum + 1);
}
```

The MH-Z19 checksum sums bytes 1 through 7, subtracts from ‘0xFF‘, then adds 1. This produces the final checksum byte.

Sending commands

```
void Mhz19::sendCommand(uint8_t cmd, uint8_t d0) {
    uint8_t frame[9] = {0xFF, 0x01, cmd, d0, 0, 0, 0, 0, 0};
    frame[8] = checksum(frame);
    _ser.write(frame, sizeof(frame));
}
```

This builds a 9-byte command frame.

‘sizeof(frame)‘ returns 9 because ‘frame‘ is a local array of 9 bytes.

‘_ser.write(frame, sizeof(frame))‘ writes the whole byte array to the sensor.

Requesting a CO2 reading

```
void Mhz19::requestReading() {
    _idx = 0;
    sendCommand(CMD_READ_CO2);
}
```

Reset the receive buffer index, then send the read command.

ABC and zero calibration

```
void Mhz19::setAutoCalibration(bool enabled) {
    sendCommand(CMD_ABC, enabled ? 0xA0 : 0x00);
}
```

If ‘enabled‘ is true, send ‘0xA0‘; otherwise send ‘0x00‘.

```
void Mhz19::calibrateZero() {
    sendCommand(CMD_ZERO_CAL);
}
```

This sends the zero calibration command. It exists but is not called by normal runtime code.

Polling responses

The parser reads bytes while available:

```
while (_ser.available()) {
    uint8_t b = (uint8_t)_ser.read();
```

It synchronizes on the response start:

```
if (_idx == 0 && b != 0xFF) continue;
if (_idx == 1 && b != CMD_READ_CO2) { _idx = (b == 0xFF) ? 1 : 0; continue; }
```

For a read response, byte 0 should be ‘0xFF‘ and byte 1 should be ‘0x86‘.

It fills the 9-byte buffer:

```
_buf[_idx++] = b;
if (_idx < 9) continue;
_idx = 0;
```

It validates the checksum:

```
if (checksum(_buf) != _buf[8]) continue;
```

Then extracts:

```
_ppm  = (int)_buf[2] << 8 | _buf[3];
_temp = (int)_buf[4] - 40;
```

CO2 ppm is stored in bytes 2 and 3 as a 16-bit value. Temperature is byte 4 minus 40, per sensor protocol.

Finally it sets ‘gotFrame = true‘ and returns it.

11. ‘src/ui.h‘

This header declares what the display module needs and exposes.

‘UiModel‘

‘UiModel‘ is a data object. ‘main.cpp‘ fills it with current sensor state, then passes it to the UI.

This separation is good design: sensor code does not need to know drawing details, and UI code does not need to know serial protocols.

Fields:

```
bool pmValid = false;
bool co2Valid = false;
bool pmWarmup = true;
bool co2Warmup = true;
```

These booleans tell the UI whether each sensor is usable or still warming up.

```
uint16_t pm1 = 0, pm25 = 0, pm10 = 0;
```

Particulate matter mass concentrations, atmospheric values, in ug/m3.

```
uint16_t n03 = 0, n05 = 0, n10 = 0;
```

```
uint16_t n25 = 0, n50 = 0, n100 = 0;
```

Particle counts by size bucket.

```
int co2 = -1;
int temp = 0;
```

CO2 ppm and rough sensor temperature.

```
int aqi = -1;
int aqiPm25 = -1, aqiPm10 = -1;
```

Combined AQI and sub-index AQIs. The combined AQI is the worse of PM2.5 and PM10.

```
uint32_t uptimeS = 0;
```

Device uptime in seconds.

UI functions

```
void uiBegin();
void uiSetModel(const UiModel& m);
void uiToggleScreen();
```

- ‘uiBegin‘: initialize the TFT screen.
- ‘uiSetModel‘: store a new UI model and redraw the current screen.
- ‘uiToggleScreen‘: switch between dashboard and details screens.

12. ‘src/ui.cpp‘

This file owns the display and all drawing code.

```
#include "ui.h"
#include <TFT_eSPI.h>
#include "aqi.h"
```

It includes its public header, the TFT library, and AQI helpers.

Private module state

Inside the anonymous namespace:

```
TFT_eSPI tft;
```

This creates the display object.

```
enum class Screen : uint8_t { DASHBOARD, DETAILS };
Screen screen = Screen::DASHBOARD;
```

‘enum class‘ is a scoped enum. Values must be written as ‘Screen::DASHBOARD‘, which avoids name collisions.

```
bool layoutDrawn = false;
UiModel model;
bool haveModel = false;
```

- ‘layoutDrawn‘: whether the static layout has already been drawn.
- ‘model‘: latest display data.

- ‘haveModel‘: whether ‘uiSetModel‘ has been called yet.

The layout is separated from dynamic values to reduce flicker and avoid redrawing the whole screen every update.

Palette

```
constexpr uint16_t COL_BG     = TFT_BLACK;
constexpr uint16_t COL_HEAD   = 0x18E3;
...
```

These are colors. Some use library constants, others use raw RGB565 hex values.

RGB565 packs red, green, and blue into 16 bits. Small TFT displays commonly use it because it saves RAM and bandwidth.

Geometry constants

```
constexpr int HEAD_H = 24;
constexpr int CARD_Y = 30, CARD_H = 112, CARD_W = 152;
...
```

These fixed pixel measurements define the dashboard layout for a 320x240 landscape screen.

Because embedded displays are fixed-size, hardcoded pixel layout is normal. This is closer to drawing on a canvas than building responsive HTML.

‘drawCardFrame‘

```
void drawCardFrame(int x, int y, int w, int h)
```

Draws a rounded rectangle card background and border.

```
tft.fillRoundRect(x, y, w, h, 8, COL_CARD);
tft.drawRoundRect(x, y, w, h, 8, COL_BORDER);
```

The ‘8‘ is the corner radius.

‘drawPill‘

This draws the colored category label at the bottom of a card.

It calculates a rectangle inside the card, fills a rounded capsule, centers text, and draws the text in black.

Important TFT concepts:

```
tft.setTextDatum(MC_DATUM);
```

This sets the text anchor point. ‘MC_DATUM‘ means middle-center.

```
tft.setTextColor(TFT_BLACK, color);
```

This sets foreground and background color. Passing the background color helps erase old text cleanly.

‘aqiToGaugeX‘

This maps an AQI value from 0-500 to an x-coordinate on the gauge.

The gauge has six segments, one for each AQI category.

```
static const int lo[6] = {0, 51, 101, 151, 201, 301};
static const int hi[6] = {50, 100, 150, 200, 300, 500};
```

For the category containing the current AQI, it interpolates horizontally within that segment.

```
aqi = constrain(aqi, 0, 500);
```

‘constrain‘ is an Arduino helper that clamps a value to a range.

‘drawHeader‘

Draws the top bar and title.

```
tft.fillRect(0, 0, 320, HEAD_H, COL_HEAD);
```

This fills a 320-pixel-wide header strip.

```
tft.setTextDatum(ML_DATUM);
```

‘ML_DATUM‘ means middle-left text anchor.

‘drawStatus‘

This draws the status text and colored dot in the header.

The logic is:

- If either sensor is warming up: show ‘WARM-UP‘ in yellow.
- Else if either sensor is invalid: show ‘NO DATA‘ in red.
- Else show ‘LIVE‘ in green.

```
tft.setTextPadding(tft.textWidth("WARM-UP", 2));
```

Text padding makes new text overwrite the same width as the longest possible status string. This avoids leaving old characters on screen.

‘drawDashboardLayout‘

This draws static dashboard elements:

- black background
- header
- AQI and CO2 cards
- labels
- six-color AQI gauge
- three PM tiles

It does not draw live values. Those are drawn by ‘drawDashboard‘.

This split is important on small displays because drawing is slow compared with normal desktop UI rendering.

‘drawGaugeMarker‘

This erases the old marker area:

```
tft.fillRect(GX - 6, MARK_Y, GW + 12, MARK_H, COL_BG);
```

Then, if PM data is valid, it draws a white triangle under the gauge at the current AQI position.

‘drawDashboard‘

This updates dynamic dashboard values.

For the AQI card:

- If PM data is valid, draw the AQI number and category pill.
- Otherwise draw ‘--‘ and a ‘WARM-UP‘ or ‘NO DATA‘ pill.

For the CO2 card:

- If CO2 data is valid, draw ppm and comfort label.
- Otherwise draw ‘--‘ and warm-up/no-data state.

For the PM tiles:

- PM1.0 is drawn in normal value color.
- PM2.5 is colored by PM2.5 AQI category.
- PM10 is colored by PM10 AQI category.

This gives the user more information than the combined AQI alone.

‘drawDetailsLayout‘

This draws the static labels for the details screen:

- PM1.0, PM2.5, PM10
- particle counts
- CO2
- sensor temperature
- AQI subindices
- uptime

The title includes ‘(’d’ = back)‘ because pressing ‘d‘ in the serial monitor toggles screens.

‘drawDetailRow‘

```
void drawDetailRow(uint8_t row, const String& value, uint16_t color = COL_VALUE)
```

This draws a right-aligned value for a given row.

‘String‘ is Arduino’s dynamic string class. It is convenient here because the details screen builds values like:

```
String(model.aqiPm25) + " / " + String(model.aqiPm10)
```

The default color is ‘COL_VALUE‘, but callers can pass another color.

‘drawDetails‘

This updates the dynamic values on the details screen.

If PM data is valid, it draws all PM and particle count rows. Otherwise it draws ‘--‘.

If CO2 data is valid, it draws CO2 and temperature. Otherwise it draws ‘--‘.

It formats uptime manually:

```
snprintf(buf, sizeof(buf), "%lu:%02lu:%02lu", hours, minutes, seconds);
```

‘snprintf‘ writes formatted text into a fixed character buffer safely, limited by the buffer size.

‘render‘

```
void render() {
    if (!layoutDrawn) {
        ...
    }
    if (!haveModel) return;
    ...
}
```

‘render‘ is the central drawing function.

If the layout has not been drawn, it draws the static layout for the current screen. If no model has been received yet, it stops there.

Once a model exists, it draws the dynamic content for the active screen.

Public UI functions

```
void uiBegin() {
    tft.init();
    tft.setRotation(1);
    render();
}
```

Initialize the display, rotate it to landscape, and draw initial layout.

```
void uiSetModel(const UiModel& m) {
    model = m;
    haveModel = true;
    render();
}
```

Copy the new model, mark data as available, and redraw.

```
void uiToggleScreen() {
    screen = (screen == Screen::DASHBOARD) ? Screen::DETAILS : Screen::DASHBOARD;
    layoutDrawn = false;
    render();
}
```

Switch screens using a ternary expression. Mark layout as not drawn so the next render clears and redraws the full screen.

13. ‘src/main.cpp‘

This is the coordinator. It wires sensors, timings, UI, button input, and serial logs together.

Header comment

The top comment explains a very important ESP8266 limitation:

Both sensors are UART devices at 9600 baud, but the ESP8266 has only one full hardware UART available, and this project keeps it for USB logging/flashing.

So the sensors use ‘SoftwareSerial‘, which is "bit-banged" serial. Bit-banged serial is implemented in software, so it is timing-sensitive and can reliably listen to only one port at a time.

That is why the firmware uses a state machine:

```
PMS_LISTEN -> CO2_WAIT -> IDLE -> next cycle
```

It listens to the particle sensor, then asks the CO2 sensor, then waits until the next 5-second cycle.

The comment also explains why the TFT is redrawn during hand-off moments. Display SPI writes can be long bursts, and those bursts could interfere with SoftwareSerial timing if they happen while receiving PMS bytes.

Includes

```
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <SoftwareSerial.h>
```

Framework/library includes:

- ‘Arduino.h‘: Arduino basics.
- ‘ESP8266WiFi.h‘: used to turn Wi-Fi off.
- ‘SoftwareSerial.h‘: software UART implementation.

```
#include "config.h"
#include "pms5003.h"
#include "mhz19.h"
#include "aqi.h"
#include "ui.h"
```

Local project headers.

Global objects

```
SoftwareSerial pmsSerial;
SoftwareSerial mhzSerial;
Pms5003 pms(pmsSerial);
Mhz19 mhz(mhzSerial);
```

Two software serial ports are created, then passed into the sensor driver objects.

The drivers store references to these serial objects. That means when ‘pms‘ reads ‘_ser‘, it is reading from ‘pmsSerial‘.

State machine

```
enum class Phase : uint8_t { PMS_LISTEN, CO2_WAIT, IDLE };
Phase phase = Phase::IDLE;
```

The firmware can be in one of three phases:

- ‘PMS_LISTEN‘: listen for a PMS5003 frame.
- ‘CO2_WAIT‘: wait for an MH-Z19E response.
- ‘IDLE‘: wait until the next sample cycle.

```
uint32_t phaseStart = 0;
uint32_t cycleStart = 0;
```

These store timestamps from ‘millis()‘.

‘millis()‘ returns milliseconds since boot. It is the standard Arduino way to do non-blocking timing.

Stored sensor state

```
PmsData pmData{};
uint32_t pmLastOkMs = 0;
bool pmEverValid = false;
```

‘pmData‘ stores the latest PMS reading. ‘pmLastOkMs‘ stores when the last valid frame arrived. ‘pmEverValid‘ tracks whether there has ever been a valid reading.

The CO2 equivalents are:

```
int co2Ppm = -1;
int co2Temp = 0;
uint32_t co2LastOkMs = 0;
bool co2EverValid = false;
```

‘startCycle‘

```
void startCycle() {
    cycleStart = phaseStart = millis();
```

Set both cycle and phase start times to now.

```
mhzSerial.enableRx(false);
```

Turn off MH-Z19E receive, because the firmware is about to listen to PMS.

```
pms.reset();
```

Drop any partial PMS frame.

```
pmsSerial.enableRx(true);
```

Start listening to the PMS serial port.

```
while (pmsSerial.available()) pmsSerial.read();
```

Flush stale bytes from the receive buffer.

```
phase = Phase::PMS_LISTEN;
```

Move the state machine into PMS listening mode.

‘startCo2Request‘

```
pmsSerial.enableRx(false);
```

```
mhzSerial.enableRx(true);
```

Stop listening to PMS, start listening to MH-Z19E.

```
while (mhzSerial.available()) mhzSerial.read();
```

Flush stale CO2 bytes.

```
mhz.requestReading();
```

Send the MH-Z19E read command.

```
phaseStart = millis();
phase = Phase::CO2_WAIT;
```

Record the start time and move to CO2 wait phase.

‘publishReadings‘

This function builds a ‘UiModel‘, sends it to the UI, and logs a summary over USB serial.

```
uint32_t now = millis();
UiModel m;
```

Create a fresh model. Its default field values come from ‘UiModel‘ defaults in ‘ui.h‘.

Warm-up flags:

```
m.pmWarmup  = now < PMS_WARMUP_MS;
m.co2Warmup = now < MHZ_WARMUP_MS;
```

Because ‘now‘ is milliseconds since boot, this checks whether boot time is still inside the warm-up window.

Validity flags:

```
m.pmValid  = pmEverValid && !m.pmWarmup && (now - pmLastOkMs) < DATA_STALE_MS;
m.co2Valid = co2EverValid && !m.co2Warmup && (now - co2LastOkMs) < DATA_STALE_MS;
```

A sensor is valid only if:

1. It has produced at least one good reading.
2. It is not warming up.
3. The latest good reading is not stale.

Uptime:

```
m.uptimeS = now / 1000;
```

Convert milliseconds to seconds.

If PM has ever been valid:

```
m.pm1 = pmData.pm1_atm;
m.pm25 = pmData.pm25_atm;
m.pm10 = pmData.pm10_atm;
...
m.aqiPm25 = aqiFromPm25(pmData.pm25_atm);
m.aqiPm10 = aqiFromPm10(pmData.pm10_atm);
m.aqi = max(m.aqiPm25, m.aqiPm10);
```

The UI gets the last PM readings even if currently warming up/stale. The ‘pmValid‘ flag controls whether to display them as live values.

The combined AQI is the worse of the PM2.5 and PM10 AQI values.

If CO2 has ever been valid:

```
m.co2 = co2Ppm;
m.temp = co2Temp;
```

Then:

```
uiSetModel(m);
```

Send the model to the UI and redraw.

Finally:

```
Serial.printf(...)
```

Print a debug line. This is like ‘console.log‘, but over the USB serial monitor.

‘setup‘

```
Serial.begin(DEBUG_BAUD);
delay(50);
Serial.println(...);
```

Start USB serial logging, wait briefly, and print a banner.

```
WiFi.mode(WIFI_OFF);
```

Turn Wi-Fi off. Even though ESP8266 has Wi-Fi, this version does not use it. Turning it off saves power and reduces heat.

```
pinMode(PIN_BUTTON, INPUT_PULLDOWN_16);
```

Configure the button pin as input with GPIO16’s internal pulldown.

```
pmsSerial.begin(PMS_BAUD, SWSERIAL_8N1, PIN_PMS_RX, -1);
```

Start PMS SoftwareSerial:

- baud: 9600
- mode: 8 data bits, no parity, 1 stop bit
- RX pin: ‘PIN_PMS_RX‘
- TX pin: ‘-1‘, meaning no transmit pin

```
mhzSerial.begin(MHZ_BAUD, SWSERIAL_8N1, PIN_MHZ_RX, PIN_MHZ_TX);
mhzSerial.enableRx(false);
```

Start MH-Z19E serial with both RX and TX, then disable RX until it is needed.

```
uiBegin();
```

Initialize the display.

```
mhz.setAutoCalibration(MHZ19_ABC_ENABLED);
```

Send the ABC configuration command to the CO2 sensor.

```
startCycle();
```

Begin the first sensor cycle.

‘loop‘

‘loop()‘ runs forever.

```
uint32_t now = millis();
```

Capture current time once for this loop iteration.

The main switch:

```
switch (phase) {
```

This is the state machine.

‘PMS_LISTEN‘

```
if (pms.poll()) {
    pmData = pms.data();
    pmLastOkMs = now;
    pmEverValid = true;
    startCo2Request();
}
```

If the PMS parser found a valid frame, save it, mark it valid, then move on to CO2.

```
else if (now - phaseStart > PMS_LISTEN_TIMEOUT_MS) {
    startCo2Request();
}
```

If no PMS frame arrived within 3.5 seconds, still try CO2. This prevents one sensor failure from blocking the whole loop.

‘CO2_WAIT‘

```
if (mhz.poll()) {
    co2Ppm = mhz.ppm();
    co2Temp = mhz.temperature();
    co2LastOkMs = now;
    co2EverValid = true;
    mhzSerial.enableRx(false);
    publishReadings();
    phase = Phase::IDLE;
}
```

If a valid CO2 response arrives, save it, stop receiving on that port, publish the readings, then enter idle.

```
else if (now - phaseStart > MHZ_RESPONSE_TIMEOUT_MS) {
    mhzSerial.enableRx(false);
    publishReadings();
    phase = Phase::IDLE;
}
```

If CO2 times out, still publish the latest available readings and enter idle.

‘IDLE‘

```
if (now - cycleStart >= SAMPLE_INTERVAL_MS) startCycle();
```

Wait until 5 seconds have passed since the start of the cycle, then begin another one.

Button and serial screen toggle

After sensor handling, the loop checks input:

```
static bool btnPrev = false;
static uint32_t btnLastMs = 0;
```

‘static‘ local variables keep their values between function calls. This is like state stored outside the function, but scoped inside the function.

```
bool btn = digitalRead(PIN_BUTTON) == HIGH;
```

Read the button.

```
if (btn && !btnPrev && now - btnLastMs > 250) {
    btnLastMs = now;
    uiToggleScreen();
}
```

This detects a rising edge: the button is pressed now but was not pressed before. The 250 ms lockout is debounce, preventing one physical press from being counted multiple times.

```
btnPrev = btn;
```

Store current state for the next loop.

Serial shortcut:

```
if (Serial.available()) {
    char c = (char)Serial.read();
    if (c == ’d’ || c == ’D’) uiToggleScreen();
}
```

If the user types ‘d‘ or ‘D‘ in the serial monitor, toggle the display screen.

```
yield();
```

On ESP8266, ‘yield()‘ lets background system tasks run. It helps avoid watchdog resets during long loops.

14. Runtime Data Flow

The full runtime flow is:

1. ‘setup()‘ starts serial logging, disables Wi-Fi, configures pins, starts SoftwareSerial ports, initializes the UI, configures CO2 ABC, and starts the first sensor cycle.
2. ‘loop()‘ enters ‘PMS_LISTEN‘.
3. ‘Pms5003::poll()‘ reads PMS bytes until it finds a valid 32-byte frame.
4. ‘main.cpp‘ saves ‘PmsData‘ and switches to CO2.
5. ‘Mhz19::requestReading()‘ sends a 9-byte command.
6. ‘Mhz19::poll()‘ waits for a valid 9-byte response.
7. ‘publishReadings()‘ builds a ‘UiModel‘.

8. ‘aqiFromPm25‘ and ‘aqiFromPm10‘ compute AQI subindices.
9. ‘uiSetModel()‘ stores the model and redraws the active screen.
10. The system idles until the 5-second interval is up.

15. Why the Code Is Split This Way

The split is clean and intentional:

- ‘main.cpp‘ owns orchestration and timing.
- ‘config.h‘ owns hardware/timing settings.
- ‘pms5003.*‘ owns PMS5003 protocol parsing.
- ‘mhz19.*‘ owns MH-Z19E protocol commands and parsing.
- ‘aqi.*‘ owns AQI domain logic.
- ‘ui.*‘ owns display rendering.

This is similar to separating a web app into:

- config
- API clients
- business/domain logic
- state orchestration
- UI components

The difference is that firmware code must also manage hardware timing, byte-level protocols, limited RAM, and display redraw cost.

16. Important Firmware Patterns Used

Non-blocking state machine

The code does not sit inside long ‘delay()‘ calls waiting for sensors. Instead, it checks available bytes and timeouts on each ‘loop()‘ pass.

This keeps the firmware responsive to button presses and serial input.

Validate before trusting sensor data

Both sensor drivers check frame shape and checksum before updating stored data. This is important because serial bytes can be corrupted.

Use timestamps instead of blocking waits

The code uses:

```
now - phaseStart > TIMEOUT
```

This is the standard Arduino pattern for timeouts.

It is also safer than comparing absolute future times because ‘millis()‘ will eventually overflow. Unsigned subtraction keeps working across overflow for interval checks.

Separate data from rendering

‘UiModel‘ decouples measurement state from drawing. This makes the UI code easier to change without touching sensor protocol code.

Redraw static layout separately

The UI draws static frames/labels only when the screen changes. Live values are updated separately. This reduces flicker and improves performance.

17. Things to Know or Watch

‘aqiCategory(-1)‘ would return Good

The helper assumes it receives a valid AQI. If called with ‘-1‘, it returns ‘AQI_GOOD‘ because ‘-1 <= 50‘.

Current callers avoid this by checking validity first. If future code calls ‘aqiCategory‘ directly, it should guard against negative AQI values.

‘main.cpp‘ banner says ILI9341

The setup banner says:

```
AQI Monitor v0.1.0 (PMS5003 + MH-Z19E + ILI9341)
```

But ‘platformio.ini‘ now defaults to ST7789. This looks like a stale log string and could be updated to avoid confusion.

‘ui.h‘ comment mentions future button

‘ui.h‘ says screens are switched with ‘d‘ and future push button on D0. But ‘main.cpp‘ already implements the D0 push button. This comment is stale.

‘String‘ in embedded code

‘ui.cpp‘ uses Arduino ‘String‘ in the details screen. That is convenient and fine for this small use, but dynamic strings can fragment memory in long-running embedded programs if heavily used. This code uses them lightly.

Instantaneous AQI approximation

The AQI comments correctly note that EPA breakpoints are defined for 24-hour averages. This firmware applies them to current sensor readings as a consumer monitor approximation.

18. Mental Model for a Web/Python Developer

Think of the project like this:

```
config.h       = constants/settings module
pms5003.cpp    = binary API client for PMS sensor
mhz19.cpp      = binary API client for CO2 sensor
aqi.cpp        = domain/business logic
ui.cpp         = canvas-style UI renderer
main.cpp       = app controller/event loop
platformio.ini = package/build/deployment config
```

Instead of HTTP requests, the "APIs" are UART byte streams.

Instead of React state, ‘UiModel‘ is a small manually updated state object.

Instead of the browser rendering DOM, ‘ui.cpp‘ paints pixels onto a TFT screen.

Instead of async/await, the firmware uses a manual state machine driven by ‘millis()‘ and ‘poll()‘ methods.

19. Glossary

- AQI: Air Quality Index.
- Baud: serial communication speed, roughly bits per second.
- Bit-banged: a protocol implemented in software timing instead of dedicated hardware.
- Checksum: a small computed value used to detect corrupted data.
- GPIO: general-purpose input/output pin.
- Header file: a ‘.h‘ file that declares functions/classes/types.
- RGB565: 16-bit color format used by small displays.
- SPI: fast bus commonly used for displays.
- TFT: thin-film transistor display.
- UART: serial communication protocol used by the sensors.